Documentación Ejercicio 5 (Práctica 1)

EscrowMarket: Compraventa con depósito en garantía sobre Ethereum

Pablo Blanco Pumar Iván Martínez Fernández

Octubre de 2026

1. Análisis y definición del escenario

El contrato inteligente **EscrowMarket** resuelve el problema de la falta de confianza al realizar compraventas entre desconocidos por Internet. Habitualmente, se requiere un intermediario centralizado (plataformas, marketplaces) que custodia el dinero a cambio de altas comisiones y con falta de transparencia.

El uso de una cadena de bloques pública como Ethereum garantiza un entorno seguro, inmutable y sin permisos. El comprador bloquea el pago en el contrato (actuando como depósito en garantía), el cual solo se libera al vendedor cuando se confirma la recepción del producto. Para los casos de conflicto, la red incluye la figura de un árbitro descentralizado. De este modo, ninguna de las partes puede apropiarse indebidamente de los fondos de forma unilateral ni manipular las reglas.

2. Diseño

2.1. Caso de uso

El sistema facilita transacciones de compraventa seguras. Un usuario crea un trato indicando la dirección del vendedor, el hash criptográfico de la descripción del producto y un plazo máximo de envío, depositando en ese instante el importe. El vendedor debe marcar el envío en plazo. Una vez que el comprador recibe el paquete, lo confirma en el contrato, lo que habilita al vendedor para retirar sus fondos. Si hay discrepancias durante la ventana de confirmación, cualquiera de las partes puede abrir una disputa, permitiendo al árbitro decidir el destino final de los fondos a cambio de una pequeña comisión.

Reglas de operación: Comprador, vendedor y árbitro deben ser tres direcciones distintas y el vendedor no puede ser la dirección cero. El importe debe ser mayor que 0. Cada trato termina en un estado final (`Completed`, `Refunded` o `Resolved`) y solo se acredita dinero al entrar en uno de ellos. Por eso el dinero de un trato no puede acreditarse dos veces.

2.2. Contenido

El contrato `EscrowMarket.sol` almacena toda la lógica de negocio y los saldos de los participantes.

Estructuras de datos:

- **Deal (Trato):** Estructura que almacena los detalles de cada compra, incluyendo el comprador, vendedor, importe, estado actual de la transacción, plazos límite y el hash de la descripción del producto.

Mapeos (Mappings):

- `deals`: Relaciona un identificador numérico único con la información de cada trato.
- `credits`: Almacena el saldo (en wei) que cada usuario tiene acreditado y disponible para retirar de forma segura.

Funciones principales:

- `createDeal`: Permite al comprador iniciar el trato depositando los fondos.
- `markShipped`: Utilizada por el vendedor para confirmar el envío del paquete.
- `confirmReceipt`: Permite al comprador confirmar la recepción, acreditando el pago al vendedor.
- `openDispute` y `resolveDispute`: Mecanismos para gestionar conflictos mediante el árbitro.
- `withdraw`: Permite a los usuarios retirar sus fondos acumulados.

Eventos:

- `DealCreated`: Se emite al crear un nuevo trato.
- `DealStateChanged`: Se emite cada vez que un trato cambia de estado.
- `DisputeResolved`: Se emite cuando el árbitro resuelve una disputa.
- `Withdrawal`: Se emite cuando un usuario retira sus fondos.

Reglas y control de acceso:

- El contrato utiliza la teoría de **Modificadores** (`inState`, `onlyBuyer`, `onlySeller`, `onlyArbiter`) para el control de acceso. Esto permite agrupar las sentencias `require()` con mensajes de error descriptivos, evitando la repetición de código en las funciones principales y manteniendo una lógica limpia y segura.
- Se emplean transiciones de estado estrictas (Funded, Shipped, Completed, Disputed, Resolved).

2.3. Usuarios del sistema

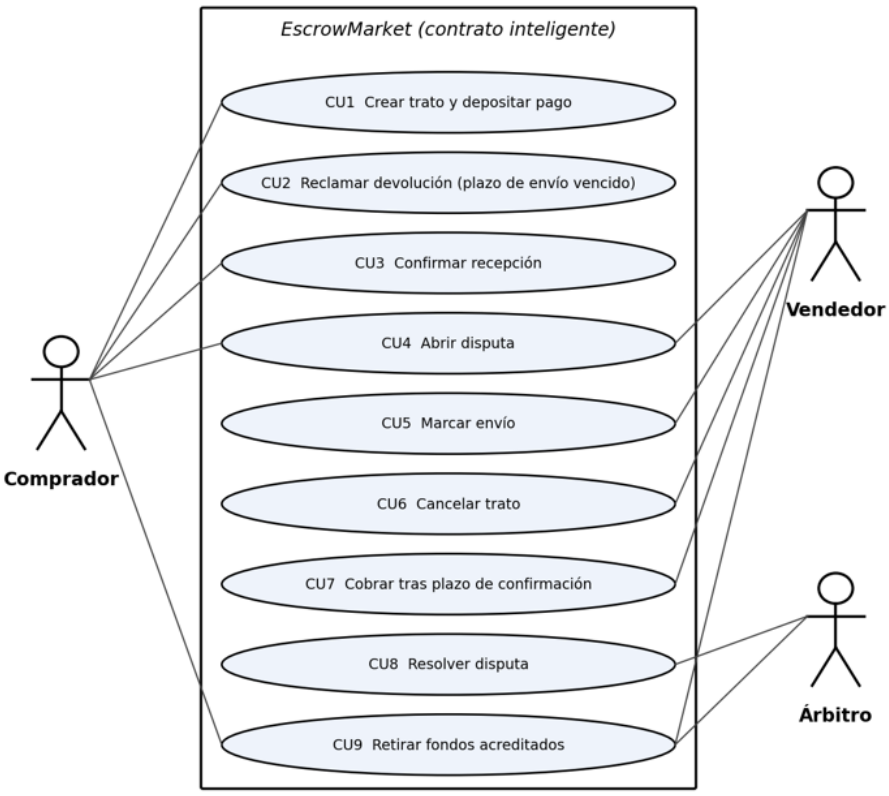


Figura 1: Diagrama de casos de uso de EscrowMarket.

- **Comprador**: Crear trato y depositar pago, reclamar devolución (plazo de envío vencido), confirmar recepción, abrir disputa, retirar fondos acreditados.
- **Vendedor**: Marcar envío, cancelar trato, cobrar tras plazo de confirmación, abrir disputa, retirar fondos acreditados.
- **Árbitro**: Dirección fijada en el despliegue (inmutable) que se encarga de resolver disputa.

2.4. Máquina de estados

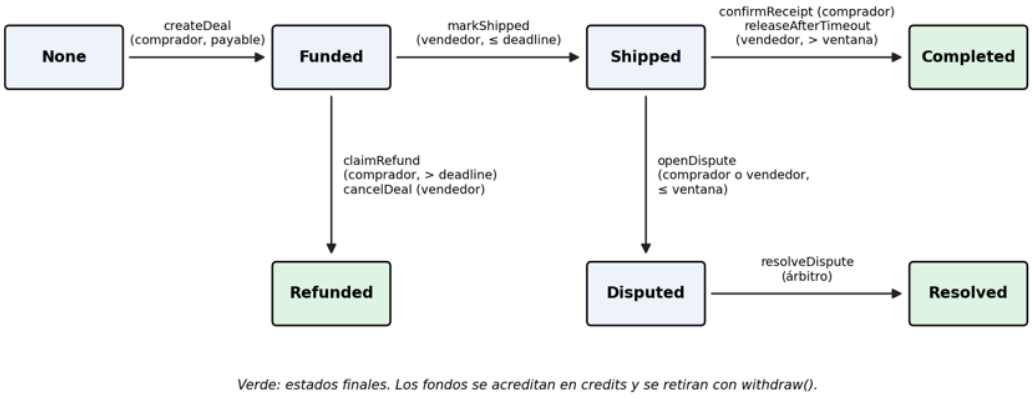


Figura 2: Máquina de estados de un trato en EscrowMarket.

3. Implementación

El código se ha desarrollado en Solidity (versión 0.8.30) utilizando el entorno Remix IDE, aplicando técnicas de optimización y seguridad.

- **Optimización de Gas (Empaquetado de variables):** Se han utilizado tipos de datos reducidos (`uint96` para el importe y `uint40` para las fechas) dentro de la estructura `Deal`. Esto permite que las variables compartan el mismo bloque o *slot* de memoria de 32 bytes de la EVM, disminuyendo enormemente las costosas operaciones de escritura. Esta es una técnica avanzada de optimización de gas basada en los conceptos impartidos en el curso CryptoZombies.

- **Patrón Pull-Payment:** Las transiciones de estado del contrato nunca envían Ether de forma automática. En su lugar, acreditan el saldo en el mapping `credits`. Cada usuario debe invocar la función `withdraw()`, que utiliza el método `.call{value: amount}()` para el envío de Ether. Este diseño previene vulnerabilidades de reentrancia y evita que un contrato malicioso bloquee el sistema al rechazar transferencias entrantes.

- **Seguridad Aritmética:** A partir de la versión 0.8.0, Solidity incorpora de forma nativa la comprobación de desbordamientos (*overflow*/*underflow*). Por ello, la seguridad de las sumas matemáticas de los saldos se delega directamente en el compilador, revirtiendo la transacción en caso de fallo sin necesidad de usar librerías externas.

4. Pruebas

A la hora de probar el código, se debe utilizar el entorno Remix IDE compilando el contrato con la versión 0.8.30.

1. **Despliegue:** Se selecciona el entorno Remix VM. Se despliega `EscrowMarket` pasándole en el constructor la dirección de una cuenta (Cuenta1) como árbitro y estableciendo una ventana de confirmación en segundos.

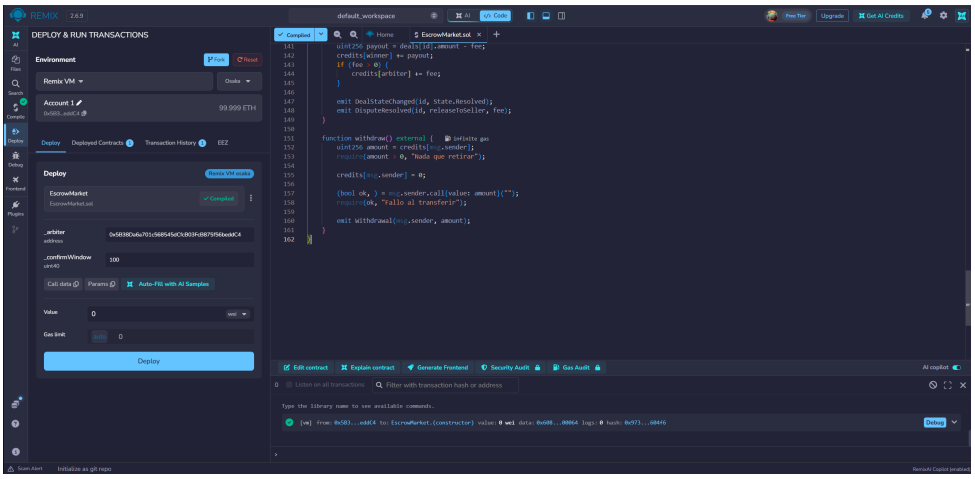


Figura 3: Compilación y despliegue del contrato.

2. **Creación de trato:** Seleccionando una cuenta de comprador (Cuenta 3), se ejecuta `createDeal` indicando el vendedor (Cuenta 2), un `bytes32` como hash y enviando Ether a través del campo "Value"superior. El contrato registrará el trato.

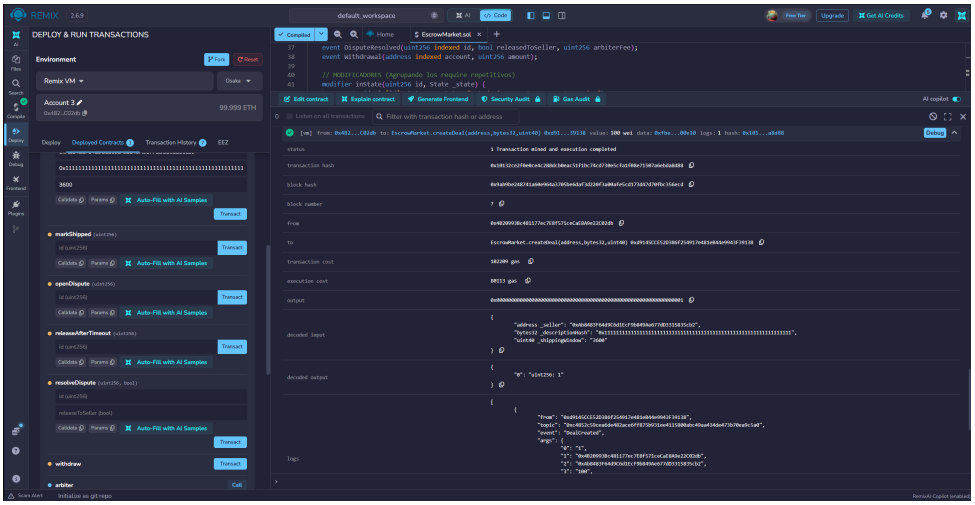


Figura 4: El comprador ejecuta `createDeal` y se emite el evento `DealCreated`.

3. **Desarrollo de la transacción:** Alternando las cuentas activas en Remix, el vendedor (Cuenta 2) ejecutará `markShipped` indicando el ID del trato. Posteriormente, el comprador (Cuenta 3) ejecutará `confirmReceipt`, lo que dejará el trato Completado y acreditará los fondos al vendedor.

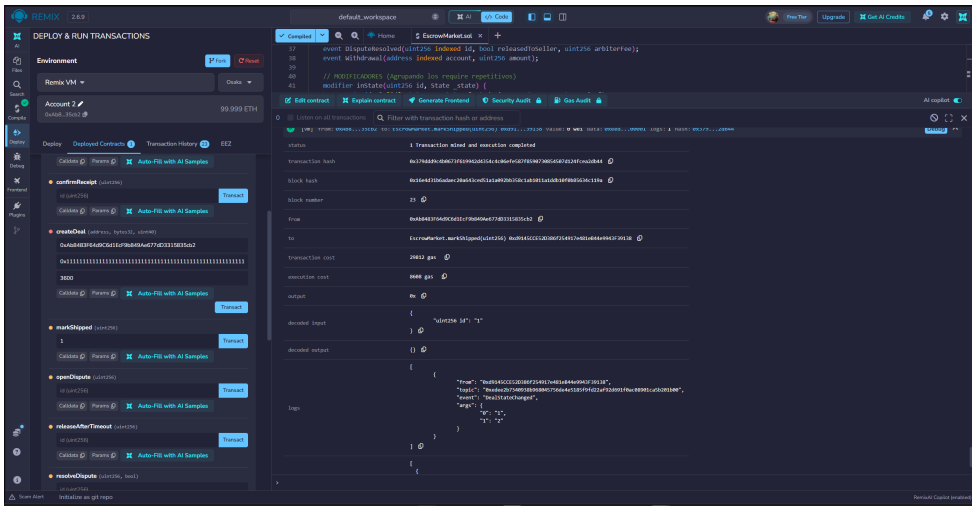


Figura 5: El vendedor ejecuta `markShipped` y el estado cambia a `Shipped`.

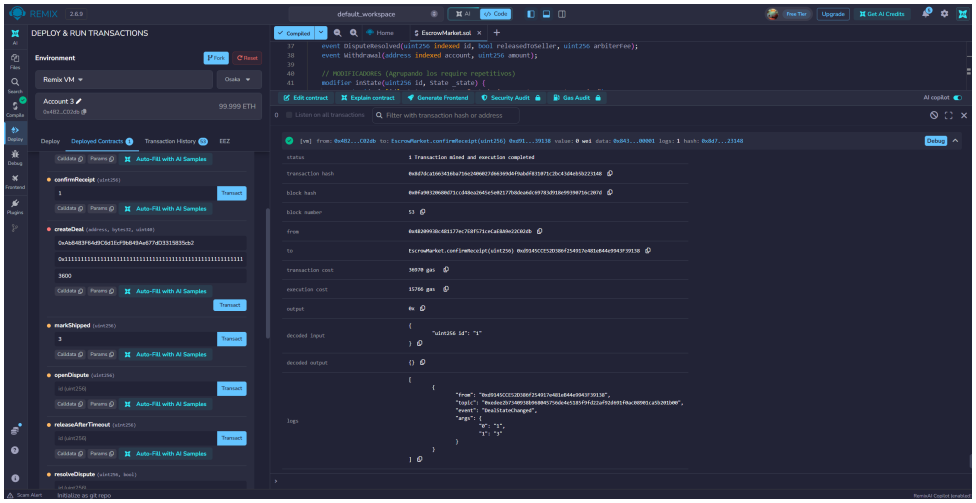


Figura 6: El comprador ejecuta `confirmReceipt`.

4. **Retirada:** Finalmente, el vendedor (Cuenta 2) invoca la función `withdraw()` sin parámetros. Se observará en el balance de su cuenta de Remix que el Ether ha sido transferido correctamente, finalizando el caso de uso con éxito.

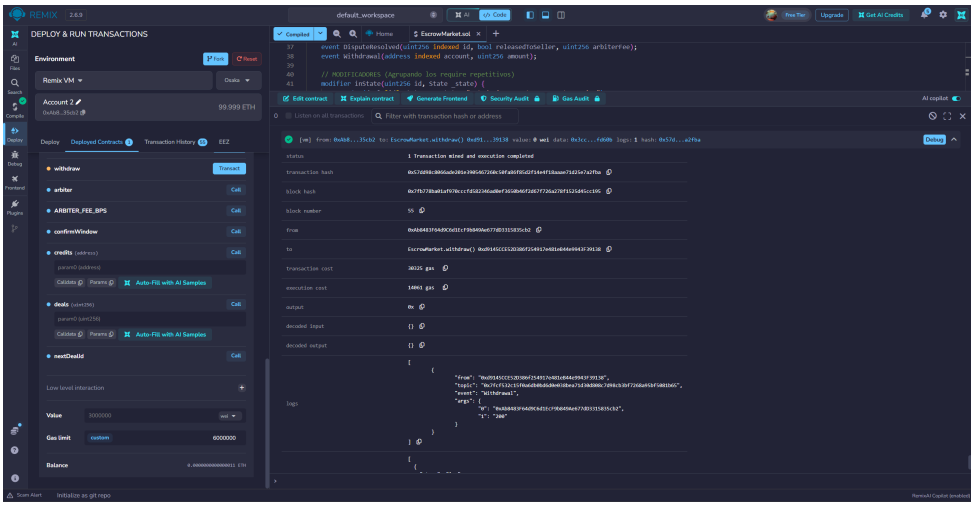


Figura 7: El vendedor ejecuta `withdraw()` para recibir el Ether en su cuenta.